

ATIVIDADE PRÁTICA 01 — ALGORITMOS EM GRAFOS

1. INTRODUÇÃO

A presente atividade tem como objetivo desenvolver uma aplicação capaz de representar e analisar uma pequena rede utilizando conceitos de grafos estudados na disciplina de Algoritmos em Grafos.

Para a elaboração do programa, foi escolhido como contexto uma **rede de entregas dos Correios**, na qual os vértices representam casas e as arestas representam as ruas que conectam essas casas. Dessa forma, o grafo permite representar as conexões existentes entre diferentes pontos de uma região e realizar um percurso entre eles.

A aplicação foi desenvolvida em linguagem C#, utilizando uma **matriz de adjacência** para representar as conexões entre os vértices e o algoritmo de **Busca em Profundidade (DFS — Depth-First Search)** para realizar o percurso do grafo.

O programa também possui funcionalidades complementares para inserção e verificação de arestas, limpeza da matriz e preenchimento automático com valores aleatórios, permitindo realizar diferentes testes sobre a estrutura implementada.

2. CONTEXTO DO GRAFO

O contexto escolhido para a aplicação foi uma **rede de entregas dos Correios**.

Nesse cenário, cada vértice representa uma casa que pode fazer parte de uma rota de entrega, enquanto cada aresta representa uma rua que permite a conexão entre duas casas.

O grafo possui oito vértices, identificados no programa como **V1, V2, V3, V4, V5, V6, V7 e V8**. Internamente, esses vértices são representados pelos índices de 0 a 7 da matriz de adjacência.

As conexões entre as casas são representadas por arestas não direcionadas. Assim, quando existe uma rua ligando duas casas, a conexão é registrada nos dois sentidos da matriz.

Os pesos associados às arestas podem representar um valor relacionado à conexão entre as casas. Para fins de implementação e testes, o programa

utiliza valores inteiros entre 1 e 10, permitindo diferenciar as conexões existentes na matriz.

2.1 Vértices

Os oito vértices utilizados representam as casas da rede:

- V0 — Casa 1
- V1 — Casa 2
- V2 — Casa 3
- V3 — Casa 4
- V4 — Casa 5
- V5 — Casa 6
- V6 — Casa 7
- V7 — Casa 8

2.2 Arestas

As arestas representam as ruas existentes entre as casas.

Como o programa permite a inserção manual e também o preenchimento aleatório da matriz, as conexões podem variar durante os testes. Quando uma aresta é inserida entre dois vértices, o programa registra o mesmo peso nas duas posições correspondentes da matriz, caracterizando uma conexão não direcionada.

Por exemplo, uma conexão entre V1 e V2 é representada tanto pela posição correspondente a $V1 \rightarrow V2$ quanto pela posição $V2 \rightarrow V1$.

3. REPRESENTAÇÃO DO GRAFO

Para representar o grafo foi utilizada uma **matriz de adjacência**.

A matriz possui dimensão 8×8 , correspondendo aos oito vértices do grafo. Cada linha e cada coluna representa um vértice, e o valor armazenado na interseção entre uma linha e uma coluna representa a existência ou não de uma aresta entre os respectivos vértices.

O valor **0** indica que não existe uma aresta entre os dois vértices. Valores maiores que zero representam uma aresta e seu respectivo peso.

A escolha da matriz de adjacência foi adequada para o projeto porque o número de vértices é pequeno e fixo, sendo limitado a oito vértices.

Além disso, a estrutura facilita a visualização das conexões e permite verificar diretamente se existe uma aresta entre dois vértices por meio de uma posição específica da matriz.

Outro fator importante é que a matriz facilita a implementação das operações utilizadas no programa, como inserção de arestas, verificação de conexões e percurso do grafo utilizando a Busca em Profundidade.

4. ALGORITMO DE BUSCA — DFS

O algoritmo utilizado para percorrer o grafo foi a **Busca em Profundidade (DFS — Depth-First Search)**.

A DFS realiza o percurso explorando um caminho do grafo até que não existam mais vértices não visitados naquela direção. Quando isso acontece, o algoritmo retorna aos vértices anteriores e continua procurando outras conexões ainda não exploradas.

No programa, a busca é iniciada a partir de um vértice informado pelo usuário. Antes do início do percurso, os vértices são classificados como não visitados. Conforme a busca avança, o programa marca os vértices visitados e registra informações relacionadas ao percurso.

A implementação utiliza uma função recursiva denominada **Visita**, responsável por explorar os vértices conectados ao vértice atual.

Durante a execução, são utilizadas três informações principais:

- **cor** : controla o estado de visitação dos vértices;
- **pred** : registra o vértice predecessor durante o percurso;
- **d** e **t** : registram os tempos de descoberta e finalização dos vértices.

No código, os estados são representados pelas cores "branco", "azul" e "vermelho". Um vértice inicialmente está branco, passa para azul quando começa a ser explorado e torna-se vermelho após a conclusão de sua exploração.

5. DESCRIÇÃO DA IMPLEMENTAÇÃO

O programa foi desenvolvido em C# e dividido principalmente em duas classes: **Program** e **Grafo**.

A classe **Program** é responsável pela interação com o usuário e pelo menu principal da aplicação. Nela são apresentadas as opções disponíveis para manipulação do grafo.

Entre as funcionalidades disponibilizadas estão:

- encerramento do programa;
- impressão da matriz de adjacência;
- inserção manual de uma aresta;
- verificação da existência de uma aresta;
- zeramento da matriz;
- preenchimento da matriz com arestas e pesos aleatórios;
- execução da Busca em Profundidade.

A classe **Grafo** concentra as estruturas e operações relacionadas ao grafo. A principal estrutura utilizada é a variável **matrizGrafo**, declarada como uma matriz de inteiros de dimensão 8 x 8.

O método **ZeraGrafo()** percorre todas as posições da matriz e atribui o valor zero, deixando o grafo inicialmente sem conexões.

O método **InserereAresta()** recebe dois vértices e um peso. Após a validação, a conexão é registrada nos dois sentidos da matriz, mantendo a característica não direcionada do grafo.

O método **ExisteAresta()** permite verificar se existe uma conexão entre dois vértices. Para isso, o programa consulta a posição correspondente da matriz e verifica se o valor armazenado é maior que zero.

O método **PreencherMatriz()** utiliza a classe **Random** para criar conexões aleatórias entre os oito vértices. O método impede a criação de laços, ou seja, conexões de um vértice com ele mesmo, e também evita repetir uma conexão já existente.

Por fim, o método **BuscaProfundidade()** recebe o vértice inicial informado pelo usuário, inicializa as estruturas necessárias para a busca e chama o método recursivo **Visita()** para realizar o percurso.

A implementação também possui mecanismos básicos de tratamento de erros para situações como entrada de valores que não sejam numéricos ou vértices fora do intervalo permitido.

6. TESTES REALIZADOS E RESULTADOS OBTIDOS

Foram realizados testes utilizando as diferentes funcionalidades disponíveis no programa.

6.1 Teste de inicialização

Inicialmente, o programa foi executado com o grafo vazio. A matriz apresentou valor zero em todas as posições, indicando que não havia conexões entre os vértices.

Resultado: teste realizado com sucesso. O grafo é inicializado sem arestas.

6.2 Teste de inserção de arestas

Foram inseridas manualmente conexões entre diferentes vértices, informando os dois vértices e o peso da aresta.

Após a inserção, a matriz apresentou os valores correspondentes nas duas posições da conexão.

Resultado: teste realizado com sucesso. As arestas foram registradas corretamente nos dois sentidos.

6.3 Teste de verificação de aresta

Foi utilizada a opção de verificar aresta para consultar conexões existentes e inexistentes.

Quando havia uma conexão entre os vértices informados, o programa apresentou uma mensagem indicando a existência da aresta e seu peso. Para conexões inexistentes, foi apresentada uma mensagem informando que a aresta não existia.

Resultado: teste realizado com sucesso.

6.4 Teste de preenchimento aleatório

A funcionalidade de preenchimento automático foi utilizada para gerar diferentes configurações do grafo.

O programa permitiu informar a quantidade de arestas desejada e gerou conexões aleatórias, atribuindo pesos entre 1 e 10.

Resultado: teste realizado com sucesso, permitindo criar diferentes grafos para a realização dos percursos.

6.5 Teste da Busca em Profundidade

Por fim, foi realizado o teste principal da atividade: a execução da Busca em Profundidade.

O usuário informa o vértice inicial e o algoritmo inicia o percurso a partir dele. A função recursiva **Visita()** explora as conexões disponíveis e avança para os vértices ainda não visitados.

Durante o percurso, o programa apresenta as conexões exploradas, permitindo acompanhar o comportamento da DFS.

Resultado: a busca foi executada corretamente a partir do vértice inicial informado, explorando os vértices conectados e utilizando as estruturas de controle de visitação implementadas no algoritmo.

Os testes demonstraram que a representação por matriz de adjacência e o algoritmo de Busca em Profundidade funcionam de maneira integrada dentro da aplicação.

7. CONCLUSÃO

O desenvolvimento desta atividade possibilitou aplicar, de maneira prática, conceitos fundamentais de Algoritmos em Grafos, especialmente a representação por matriz de adjacência e a Busca em Profundidade.

A escolha de uma rede de entregas dos Correios permitiu relacionar os conceitos estudados com uma situação prática, na qual as casas podem ser representadas como vértices e as ruas como arestas. Essa representação demonstra como estruturas de grafos podem ser utilizadas para modelar redes e analisar as conexões existentes entre diferentes pontos.

A utilização da matriz de adjacência mostrou-se adequada para o projeto devido à pequena quantidade de vértices e à facilidade de consultar e modificar as conexões. Já a implementação da DFS permitiu compreender, na prática, como um algoritmo de busca pode explorar uma rede a partir de um ponto inicial.

Além da implementação da busca, as funcionalidades de inserção, verificação, limpeza e geração aleatória de arestas contribuíram para a realização de diferentes testes e para a validação do funcionamento do programa.

Dessa forma, a atividade contribuiu para consolidar os conhecimentos relacionados à modelagem de problemas utilizando grafos, à representação de suas conexões e à aplicação de algoritmos de percurso, demonstrando a importância desses conceitos no desenvolvimento de soluções computacionais.

8. LINK DO REPOSITÓRIO

Link: [Clique aqui](#)

9. NOME DOS INTEGRANTES

- João Vitor Alves Amaral
- Matheus Henrique Borges Ferreira